```
const x: number = TicketStatus.Held
```

TypeScript will casually allow you to assign your TicketStatus enum to a variable declared as a number, which means we need to talk about the values TypeScript gives to enums.

Assigning Enum Values

By default, TypeScript enums are numbers, starting at 0 and autoincrementing. Behind the scenes, TypeScript basically creates a lookup table matching the enum properties to their values.

If you'd like, you can explicitly specify the value of each element in the enum:

```
enum TicketStatus {
  Unsold = 1,
  Held = 2,
  Purchased = 3,
  Refunded = 4,
  Invalid = 5
}
```

I strongly recommend that if you are specifying the value for one of the enums, that you specify it for all of them. If you don't, TypeScript will autoincrement from the closest specified enum, but it's potentially surprising and hard to debug when TypeScript implicitly specifies the numbers, so you may as well specify all of them.

You can also specify your enums to have string values, so we could do this:

chapter_13/03/app/javascript/contexts/venue_types.ts

```
export enum TicketStatus {
  Unsold = "unsold",
  Held = "held",
  Purchased = "purchased",
  Refunded = "refunded",
  Invalid = "invalid",
}
```

For all intents and purposes, this is functionally identical to the string literal type